

Voice-of-Customer Analytics

Talipapa extracts shopper phrases (search and voice) into structured attributes, uses those attributes to rank products, and shows admin insights for stocking and pricing.

In-app links

Sign in as an **admin**, then open:

Page	URL
Admin dashboard	/admin/dashboard
Voice of Customer	/admin/voc

Public search (same extractor, no admin login):

Example	URL
Cheap tuyo	/products?q=barato+na+tuyo
Milk not from cow	/products?q=gatas+na+dili+sa+baka
Vegan Nestle 300ml	/products?q=gatas+na+vegan+300+ml+nestle

Sidebar: **Voice of Customer**. On the VoC page, use the playground chips to extract attributes live.

Flow

```
Search / voice / playground
  ↓
QueryAttributeExtractor (+ VoiceNlpService when OpenAI is on)
  ↓
ProductSearchService (rank, exclude, cheap boost)
  ↓
VocLogger → voc_utterances + voc_attributes
  ↓
Admin /admin/voc (VocInsightsService)
```

Example phrases

These three utterances are the v1 contract. The playground chips on `/admin/voc` use the same strings.

Phrase	Product	Brand	Size	Diet	Price	Exclusion
barato na tuyo	tuyo / dried fish	—	—	—	cheap	—
gatas na dili sa baka	milk	—	—	non-cow	—	cow
gatas na vegan 300 ml nestle	milk	Nestle	300ml	vegan	—	cow

barato na tuyo

Cheap dried fish. Keywords include `tuyo` and `dried fish`; `barato` is a price word, not a product.

Search: cheaper **Tuyo (Dried Fish) 100g** ranks above **Premium Tuyo 250g**.

gatas na dili sa baka

Milk that is not from a cow (Bisaya negation). Language is `Bisaya`. Exclusions include `cow`. Keywords omit `cow` and `baka`.

Search: Nestle Vegan Oat Milk 300ml is kept. Fresh Milk 1L (cow) is dropped.

gatas na vegan 300 ml nestle

Plant milk, branded, sized. Brand `nestle`, units `300ml`, dietary `vegan` (and usually `plant-based` / `non-cow`), exclusions `cow`.

Search: Nestle Vegan Oat Milk 300ml is first. Cow fresh milk is excluded.

Code map

All paths are relative to the project root.

Extract and attribute

`app/Services/Voc/QueryAttributeExtractor.php` — turns a phrase into product, brand, units, diet, price intent, exclusions, and `attributes[]`.

Lines	Method	What it does
19–58	<code>extract()</code>	Main entry. <code>"barato na tuyo"</code> starts here.
64+	<code>mergeParsed()</code>	Fills gaps when OpenAI returns JSON.

Lines	Method	What it does
123–156	<code>toNlpArray()</code>	Builds the NLP payload used by search and logging.
161+	<code>attributesFrom()</code>	<code>{type, value, polarity}</code> rows for analytics.
213	<code>detectPriceIntent()</code>	<code>barato / mura / cheap</code> → <code>cheap</code> .
230	<code>detectBrand()</code>	Matches brands (e.g. Nestle).
247	<code>detectDietary()</code>	<code>vegan , plant-based , non-cow</code> .
266	<code>detectExclusions()</code>	<code>dili sa baka / not from cow</code> → exclude cow.
285	<code>detectProduct()</code>	<code>tuyo , gatas</code> → canonical <code>milk</code> .

config/voc.php — stopwords, brands, cheap/premium words, dietary phrases, `tuyo` → dried fish, `baka` → cow, cow-dairy vs plant-milk terms.

config/voice-assistant.php

Lines	What it does
23	<code>nlp_max_tokens</code> default 500 (larger JSON schema).
31–54	Synonyms. VoC additions around 39–46 : <code>tuyo , vegan , oat , nestle , barato</code> .

app/Services/VoiceAssistant/VoiceNlpService.php

Lines	Method	What it does
19–28	<code>extract()</code>	Cached <code>extract (voice_nlp_v2:)</code> .
32–100	<code>extractFromApi()</code>	OpenAI JSON: product, brand, dietary, price_intent, exclusions.
104–120	<code>normalizeParsed()</code>	Merges API output with the rule extractor.
124+	<code>fallbackExtract()</code>	Used when there is no API key (tests + local).

Search that uses attributes

app/Services/VoiceAssistant/ProductSearchService.php

Lines	Method	What it does
17–30	<code>paginate()</code>	Public <code>GET /products?q=</code> .

Lines	Method	What it does
36–53	<code>searchFromNlp()</code>	Voice assistant + admin playground.
~123	<code>cheap boost</code>	Lower price scores higher when <code>price_intent = cheap</code> .
146	<code>scoreProduct()</code>	Extra points for brand, diet, unit (300ml).
203	<code>isExcluded()</code>	Drops cow milk for vegan / “dili sa baka”.
238	<code>missesCanonicalProduct()</code>	A milk query must match a milk SKU.
515	<code>formatAmount()</code>	Keeps <code>300ml</code> as 300, not 3.

`app/Http/Controllers/Public/ProductController.php`

Lines	What it does
20–25	Injects extractor + logger.
35–37	Extracts attributes from <code>q</code> .
39	Searches with that NLP payload.
41–43	Logs the utterance (<code>source = search</code>).

`app/Http/Controllers/VoiceAssistantController.php`

Lines	What it does
62–69	NLP extract, then <code>searchFromNlp()</code> .
75	Logs <code>product_search</code> / <code>add_to_cart</code> as <code>source = voice</code> .
78–90	JSON includes product, brand, dietary, exclusions, attributes.

Persist Voice-of-Customer

`database/migrations/2026_09_09_000001_create_voc_tables.php`

Lines	Table
11–23	<code>voc_utterances</code> — text, language, intent, result_count, source
25–35	<code>voc_attributes</code> — type, value, polarity

Models: `app/Models/VocUtterance.php` , `app/Models/VocAttribute.php` ,
`User::vocUtterances()` .

Enums: `VocSource` (`search` / `voice` / `playground`), `VocAttributeType` , `VocPolarity` .

`app/Services/Voc/VocLogger.php`

Lines	What it does
16–22	Skip empty / tiny queries.
24–30	Dedupe the same actor + query + source within a minute.
34–41	Insert <code>voc_utterances</code> .
45–65	Insert <code>voc_attributes</code> .
66–68	Logging errors must not break search.

Admin analytics + playground

`routes/web.php` (admin-only group)

```
180 GET /admin/voc          → admin.voc.index
181 POST /admin/voc/extract → admin.voc.extract
```

`app/Http/Controllers/Admin/VocController.php`

Lines	Method	What it does
24–40	<code>index()</code>	Date range + insights + the three example chips.
43–75	<code>extract()</code>	Playground API: extract, search, log as playground.

`app/Services/Voc/VocInsightsService.php` — `summarize()` builds KPIs, product vs SKU coverage, unmet queries, dietary/brand breakdowns, and rule-based decision cards.

File	Role
<code>resources/views/admin/voc/index.blade.php</code>	Dashboard, bars, playground, recent utterances
<code>resources/js/voc-playground.js</code>	Chip click / form → <code>POST /admin/voc/extract</code>
<code>resources/js/app.js</code>	Calls <code>initVocPlayground()</code>

File	Role
<code>resources/views/layouts/admin.blade.php</code> line 24	Sidebar nav item
<code>resources/views/admin/dashboard.blade.php</code> line 26	Shortcut tile

Demo catalog

`database/seeder/DemoDataSeeder.php`

Lines	What it seeds
53–54	Cheap Tuyo 100g (₱35) and Premium Tuyo 250g (₱95)
78	Fresh Milk 1L (cow) — so exclusion can reject it
79	Nestle Vegan Oat Milk 300ml
183	Calls <code>seedVocExamples()</code>
283+	Sample utterances so <code>/admin/voc</code> is not empty

Tests

File	What it covers
<code>tests/Unit/QueryAttributeExtractorTest.php</code>	The three phrases extract correctly.
<code>tests/Unit/ProductSearchServiceTest.php</code> (~86–185)	Cheap tuyo ranks first; cow milk excluded; vegan Nestle wins.
<code>tests/Feature/VocAnalyticsTest.php</code>	Search writes utterances; <code>/admin/voc</code> is admin-only; playground JSON.

```
php artisan test --
filter="QueryAttributeExtractorTest|ProductSearchServiceTest|VocAnalyticsTest"
```

Suggested reading order

1. `app/Services/Voc/QueryAttributeExtractor.php` — extract
2. `app/Services/VoiceAssistant/ProductSearchService.php` — rank

3. `app/Services/Voc/VocLogger.php` — persist
4. `resources/views/admin/voc/index.blade.php` — what you see